

Table of Contents

1. INTRODUCTION	3
2. Setup	4
2.1 Assumptions	4
2.2 KNN Algorithms	4
2.3 Selecting <i>k</i> and <i>r</i> Parameters	5
2.4 Performans Metrics	5
2.5 Choosing the G-Smote Algorithm	7
3. Phase 1	8
3.1 General Structure	8
3.2 SMOTE Algorithm	8
3.3 Procedures and Selected Parameters	9
3.4 Findings and evaluations	10
4. Phase 2	12
4.1 Structure of Clustering	12
4.2 Choosing the number of clusters	12
4.3 Oversampling Rule	13
4.4 Implementation	15
4.5 Findings and evaluations	16
5. Conclusion	18
6. APPENDIX	20
Appendix A. Dataset-Level Phase 1 Results	20
Appendix B. Dataset-Level Phase 2 Results	21
APPENDIX CODES	Error! Bookmark not defined.

1. INTRODUCTION

In classification problems, when there is a significant difference in the number of samples between classes, the approach used to ensure the model can correctly learn the minority class is called Imbalance Learning. Standard classifiers are trained to optimize overall accuracy, so they tend to favor one class when it is dominant. The result is a model that handles the majority class well but fails with the minority class. Distance-based classifiers, such as KNN, are no exception, as imbalance in a neighborhood tends to cause majority samples to surround a query point and leave several nearby minority samples behind in the voting.

One method used to solve this problem is to sample the minority class excessively, equaling its number of datapoints to the majority class. SMOTE is a standard method that creates new points between a minority sample and its minority neighbors. G-SMOTE generalizes this idea geometrically. Instead of generating data points along the line between two points, it defines a safe region around the minority sample and draws synthetic points from within this region. It also prevents these points from crossing into the majority region. A different method involves first clustering the minority class and oversampling within each cluster.

This project compares these strategies on ten imbalance datasets using 5 different KNN classifiers. It evaluates them using metrics that are meaningful under imbalance, namely geometric mean, error, and F1 score.

The study was conducted in two phases:

Phase1, it establishes the complete pipeline including standardization, 5fold cross-validation, classifier hyperparameter selection, and two basic oversampling methods, and compares three different methods: No Oversampling, SMOTE, and G-SMOTE.

Phase2, it adds a clustering step before oversampling. This provides a procedure designed to select the number of clusters and a distribution to the synthetic points among the clusters.

The report describes how these steps were carried out and discusses the results.

2.Setup

2.1 Assumptions

- 1- All features are numerical, and the Euclidean distance between feature vectors is assumed to be a meaningful measure of similarity. Every classifier and every oversampler in the study relies on this distance.
- 2- Because distance is scale sensitive, each characteristic is standardized to zero mean and unit variance. The mean and standard deviation are calculated only on the training portion and then applied to the test portion. This ensures that no information from the test set affects the training.
- 3- Oversampling generates the number of synthetic minority samples needed to equalize the number of data points for two classes in the training set.
- 4- For each dataset, the neighborhood size for the nearest neighbor classifiers and the radius for the radius classifier are selected once and then reused for all five oversampling settings.
- 5- Oversampling and clustering are guided by a single, fixed seed. This ensures that the same results are obtained under the same conditions.
- 6- Each dataset is divided into five folds that maintain class proportions. Each fold is used once as a test set, while the remaining four folds form the training set. The values in the report are averaged across the five folds.
- 7- It is assumed that $m=2$ in Fuzzy KNN Algorithms.

2.2 KNN Algorithms

The KNN algorithms covered in this course are listed below. The code for these algorithms was taken from Computational Assignments.

- 1- Classical KNN
- 2- Modified KNN
- 3- Fuzzy KNN Variant 1 $\{0,1\}$ ($m=2$)
- 4- Fuzzy KNN Variant 2 $[0,1]$ ($m=2$)
- 5- Radius KNN

2.3 Selecting k and r Parameters

For each of the four classifiers using the number of neighbors, a k value is required. For the radius classifier, an r value is required. These values are selected once for each dataset before Phase 1 and Phase 2 and kept constant throughout the experiment. Candidate k values range from 1 to 10, and for radius, values range from 0.5, 1.0 ... 5.0. For each candidate value, the classifier is built in the internal training phase and evaluated in the internal validation phase, and the validation error is recorded. The errors are averaged over five external iterations, and the value that yields the lowest average validation error is selected. Each of the four count-based classifiers takes its own k value, and the radius classifier takes its own r value; all of these are selected independently for each dataset. The selection is performed on the original data before any oversampling takes place. Therefore, the parameters define the fundamental behavior of each classifier in the actual class distribution. The selected values are then reused unchanged in each of the five oversampling settings in both Phase 1 and Phase 2. Only the composition of the training set changes between settings, while the classifier configuration remains constant, so any difference in performance can be attributed to oversampling, not to recalibration.

2.4 Performans Metrics

Initially, I planned to evaluate the classifiers using the error rate, which was the metric we used throughout the course. While reviewing the SMOTE and G-SMOTE papers, I realized that the error rate is not the only metric used for imbalance classification and that it alone doesn't capture what's most important in this context. This led me to use the additional metrics that these papers relied on.

There are two reasons why the error rate remains low. First, it is controlled by the majority class. When one class is much larger than the other, a classifier that predicts the majority class for almost every sample will achieve a low error rate while completely failing to observe the minority class. The error rate counts these missed minority samples, but since they are few enough to change the number, a bad minority classifier and a good classifier can produce very similar results. Secondly, oversampling reduces the weight of the majority class. Adding synthetic minority samples until the classes are balanced reduces the relative influence of the majority class in the training set, thus forcing the classifier to more easily predict the minority class. This is the intended effect and increases the number of minority samples recovered. On the other hand, it causes some majority samples to be predicted as minority. This

can keep the error rate constant or even increase it. When only the error rate is considered, this intentional shift is recorded as a loss rather than an improvement. This is another reason why comparisons between methods should be made using geometric mean and F1 score in addition to error rate.

The articles address this issue by measuring performance with quantitative methods that remain sensitive to minority groups. Therefore, comparisons between methods are made using geometric mean and F1 score in addition to error rate.

The three metrics represent the following: Error rate, representing overall accuracy, the proportion of all incorrect estimates. In the case of imbalance, it primarily reflects the majority class. The geometric mean represents the balance between correctly identifying the minority class and correctly identifying the majority class. It is only high when the classifier performs well in both simultaneously and tends towards zero if either class is neglected. Therefore, it shows whether the minority class is addressed without sacrificing the majority class. The F1 score represents the quality of the estimates made for the minority class. It compares how many of the samples marked as minority are minority to how many of the samples that are minority were identified. Therefore, it is high when minority estimates are both reliable and complete.

These measures are calculated from the confusion matrix, treating the minority class as a positive class. TP, TN, FP, and FN count the minority samples estimated as a minority, the majority samples estimated as a majority, the majority samples estimated as a minority, and the minority samples estimated as a majority, and n is their sum. When any of the denominators is zero, the corresponding quantity is considered zero.

$$\text{Error} = \frac{\text{FP} + \text{FN}}{n}$$

$$\text{Sensitivity} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

$$\text{Specificity} = \frac{\text{TN}}{\text{TN} + \text{FP}}$$

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

$$G - \text{mean} = \sqrt{\text{Sensitivity} \cdot \text{Specificity}}$$

2.5 Choosing the G-Smote Algorithm

SMOTE randomly generates a synthetic data point along a straight segment between a minority point and one of its minority neighbors. G-SMOTE retains the idea of generating new points near existing minority points but replaces this segment with a geometric region around a minority point. It creates the synthetic sample from within this region. This allows for greater variety of synthetic points while also controlling where they are allowed to fall.

The shape and layout of the region are governed by three parameters. The selection strategy determines which neighbor defines the region and how far it extends. The truncation factor can cut the region with a hyperplane and remove a portion of it. The deformation factor can flatten the region from a complete sphere to a flat segment used by SMOTE, thus returning a fully deformed region G-SMOTE to SMOTE-like behavior.

Instead of adjusting this configuration per dataset, it is fixed for the entire run; thus, G-SMOTE is represented by a single, well-justified form. A combined selection strategy without truncation and distortion has been used.

The unified selection strategy is the part that keeps the synthetic points safe. For a chosen center point, it considers one of the center's minority neighbors and the nearest majority point. It adjusts the radius of the region to whichever of the two is closer to the center. If a majority point is closer than a minority neighbor, the radius shrinks to that majority distance, so the region stops before reaching the majority area and cannot enter the generated point.

Setting truncation and deformation to zero keeps the region as a complete hypersphere. This is the way G-SMOTE differs most from SMOTE, as it spreads synthetic points across the entire safe sphere, not just along a single line. This choice was made relying on the article's finding that the best results overwhelmingly come from configurations with no deformation. Next, a point is drawn uniformly through the hypersphere of radius R around the center. Smooth drawing uses the standard technique of taking a Gaussian vector and scaling the radius to fix a smooth direction so that the points are evenly distributed within the ball rather than clustered near the center:

$$R = \min(\|x_{\text{center}} - x_{\text{min}}\|, \|x_{\text{center}} - x_{\text{maj}}\|)$$

$$x_{\text{gen}} = x_{\text{center}} + R \cdot \rho \cdot \left(\frac{v}{\|v\|} \right)$$

$$\rho = U(0,1)^{\frac{1}{p}}$$

Here, p is the number of features. The synthetic point is labeled as a minority and added to the training set. The process is repeated until the classes are balanced.

Instead of adjusting this configuration according to the dataset, it was kept constant for the entire study. A combined selection strategy without truncation and deformation was used. The article reports that the strongest results were obtained with this G-SMOTE form, i.e., the pure configuration combined with this selection strategy, and especially with data that has high imbalance. Therefore, this configuration was adopted as a fixed G-SMOTE setting for the experiments instead of tuning the truncation and deformation factors separately for each dataset.

3. Phase 1

3.1 General Structure

Phase 1 establishes the basic pipeline and uses it to compare three training settings: No Oversampling, SMOTE, and G-SMOTE. Each setting produces a training set, and each of the five classifiers is trained on this set. Performance is measured using three metrics. The goal is to answer two questions: how much does oversampling improve performance compared to training on the original unbalanced data and how do SMOTE and G-SMOTE compare to each other.

3.2 SMOTE Algorithm

G-SMOTE was explained in Section 2.5, therefore the only method to be explained is SMOTE. SMOTE rebalances the training set by creating synthetic minority samples through the random selection of a point among the available minority points. To create a synthetic sample, two different minority points x_1 and x_2 are randomly selected and the new point is placed at a random location on the straight line between them:

$$x_{\text{new}} = \lambda \cdot x_1 + (1 - \lambda) \cdot x_2, \quad \lambda \sim U(0,1)$$

A λ value close to 0 places the new point near x_2 , and a value close to 1 place it near x_1 . Thus, the synthetic point always lies between the two real minority points. This process is repeated until the minority class reaches the same size as the majority class.

3.3 Procedures and Selected Parameters

The procedures are the same for all three settings and five classifiers. The dataset is divided into five folds that maintain class ratios. Each fold is used once as a test set. The other four folds form the training set. The train set is balanced according to the analyzed oversampling. No oversampling is left unchanged. In the other two settings, balancing is done with SMOTE or G-SMOTE until the classes are equal. Oversampling is not applied to the test set. This preserves the true class distribution.

Each classifier is implemented with a fixed parameter value for that dataset. The number of neighbors k for the four-count classifier and the radius r for the radius classifier are selected using the procedure in Section 2.3. The three metrics are calculated across a test set, and the values are averaged over a five-fold.

Table 1. Selected neighbor count k and radius r per dataset

Dataset	k (KNN)	k (Modified KNN)	k (Fuzzy V1)	k (Fuzzy V2)	r (Radius)
BCWD	7	9	7	10	4
CTO	1	1	1	8	0,5
Ecoli	1	1	1	2	0,5
Glass	5	8	9	1	2,5
Haberman	8	10	9	3	5
Heart	10	4	8	2	3
PageBlock	4	6	5	1	0,5
Parkinson	3	5	3	5	2
Pima	8	10	9	10	3
Yeast	10	9	10	4	1

3.4 Findings and Conclusion

The Phase 1 results are reported in Tables 2, 3, and 4. These tables compare No Oversampling, SMOTE, and G-SMOTE across five classifiers.

The results show that oversampling improves the metrics that are more meaningful under imbalance. The average G-Mean increases from 0.7653 without oversampling to 0.8214 with SMOTE and 0.8057 with G-SMOTE. The average F1 score also increases from 0.6770 to 0.6946 with SMOTE and 0.6966 with G-SMOTE.

Table2. *Average errors of 5 Different Classifiers in 3 Different Setups*

Classifier	No Oversampling	SMOTE	G-SMOTE
KNN	0,1198	0,1402	0,1354
ModKNN	0,1248	0,1377	0,1368
FuzzyV1	0,1227	0,1384	0,1348
FuzzyV2	0,1722	0,1773	0,17
RadiusKNN	0,1234	0,139	0,1274
Average	0,1326	0,1465	0,1409

The error rate behaves differently. The lowest average error belongs to No Oversampling with 0.1326. SMOTE increases the average error to 0.1465. G-SMOTE increases it to 0.1409. This does not mean that No Oversampling is better. Error is mostly controlled by the majority class. A classifier can keep error low by predicting the majority class more often. This is why Error alone is not enough for imbalance learning.

SMOTE gives the strongest average G-Mean. This means that SMOTE produces the best balance between the two classes in Phase 1. G-SMOTE gives the strongest average F1 score. This means that G-SMOTE produces slightly better minority class prediction quality. G-SMOTE also keeps the error rate lower than SMOTE. The classifier results show that Classical KNN and Fuzzy V1 are the strongest classifiers in Phase 1. They reach the highest G-Mean and F1 values under oversampling. Fuzzy V2 is the weakest classifier. Its F1 score stays much lower than the others. Radius KNN benefits strongly from oversampling in G-Mean.

Table3. Average Geometric Mean of 5 Different Classifiers in 3 Different Setups

Classifier	No Oversampling	SMOTE	G-SMOTE
KNN	0,773	0,8397	0,8288
ModKNN	0,7775	0,834	0,8216
FuzzyV1	0,7784	0,8426	0,8255
FuzzyV2	0,774	0,7807	0,7801
RadiusKNN	0,7239	0,8102	0,7727
Average	0,7653	0,8214	0,8057

Table 4. Average F1-Score of 5 Different Classifiers in 3 Different Setups

Classifier	No Oversampling	SMOTE	G-SMOTE
KNN	0,6987	0,7152	0,7239
ModKNN	0,6991	0,7147	0,7196
FuzzyV1	0,7027	0,7217	0,724
FuzzyV2	0,6276	0,6278	0,634
RadiusKNN	0,6569	0,6937	0,6817
Average	0,677	0,6946	0,6966

The dataset level results show that the effect of oversampling is not the same for every dataset. For CTO, PageBlock, and Yeast the main gain appears in G-Mean. These datasets are also large and highly imbalanced. Therefore this behavior may be caused by size, imbalance ratio, or both. Oversampling improves class balance in these datasets. However F1 does not always improve. In PageBlock and Yeast the No Oversampling case keeps a higher F1 score than the oversampling cases. This shows an important tradeoff. Oversampling makes the classifier more willing to predict the minority class. This usually improves G-Mean. It can also create more false positive predictions. This can lower F1 even when G-Mean improves. Smaller datasets show a more favorable pattern. Ecoli, Haberman, Heart, and Pima generally benefit from oversampling in both G-Mean and F1. Haberman is the clearest example. Its No Oversampling performance is weak. SMOTE and G-SMOTE improve its minority class performance strongly.

BCWD behaves differently from the other datasets. It already has very high scores without oversampling. This means that the classes are easier to separate. Oversampling adds only a small improvement in this dataset. Overall Phase 1 shows that oversampling is useful for KNN based classifiers on imbalanced datasets. SMOTE is stronger for G-Mean. G-SMOTE is slightly stronger for F1 and error. The results also show that the success of oversampling depends on the dataset structure. (The detailed dataset-level Phase 1 results are provided in Appendix Tables A.1 to A.5.)

4. Phase 2

4.1 Structure of Clustering

Phase 2 adds a clustering step before oversampling. It does not generate synthetic points directly from the minority class. It first splits the minority into clusters. Then it oversamples inside each cluster separately. This way the generated points follow the inner structure of the minority. Generation happens inside the real subregions of the minority.

The minority class is clustered with K-means. The Euclidean distance is used. The standardized features are used. These are the same distance and scaling used across the whole study. K-means splits the points into a given number of groups. It does this by minimizing the sum of squared distances inside the clusters.

K-means needs the number of clusters in advance. But this number is not known. It also changes from one dataset to another. A minority class can be a single tight group. It can also be several separate groups. Fixing the number by hand would be arbitrary. Instead the number is chosen from the data. This choice uses the elbow method described in the next section. Clustering runs separately on each training fold. It uses only the minority points of that fold. After clustering the points are generated inside the clusters. The weight rule in Section 4.3 sets the number for each cluster.

4.2 Choosing the number of clusters

The number of clusters is chosen from the data with the elbow method. k-means is run for every candidate number from two up to an upper limit. For each number the sum of squared distances inside the clusters (SSE) is recorded. SSE always drops as the number of clusters grows. What matters is the rate of the drop. The first clusters lower SSE sharply. Once the real

groups are captured, extra clusters lower SSE only a little. The number of clusters is taken at the elbow of this curve. The elbow is the point where the sharp drop turns into a slow one.

The upper limit is the floor of the square root of the number of points to be generated. This number equals the difference between the majority and the minority. (I did not want too many single clusters to form, because g-smote requires more than one point. Therefore, I set a limit to prevent the Elbow Method from creating too many clusters. It may not be a statistically significant limit, but it is quite practical because it produces reasonable results.) The upper limit is therefore tied to the amount of oversampling. The lower limit is two. This is because Phase 2 must always cluster. If the curve gives no clear elbow, the number drops to two. This happens when the inner structure of the minority is weak. In the first fold the chosen number was four on small sets such as Glass and Parkinson. It was fourteen on large sets such as CTO and PageBlock.

4.3 Oversampling Rule

After clustering the synthetic points are shared among the clusters. The sharing is not equal. Each cluster gets points in proportion to a weight. This weight sends the points to the most useful and safest regions. The weight is the product of four factors. Each factor is a ratio between zero and one. The weights are then normalized across the clusters.

The first factor is the ratio of the cluster size to the largest cluster size. Larger clusters therefore get more points. The reason is to preserve the inner distribution of the minority. A large cluster is an established part of the minority. Giving it a proportional share keeps the relative sizes of the subgroups. If every cluster were filled equally, a small group would swell too much. This would shift the distribution toward its rarest corners.

The second factor measures the tightness of the cluster. For each cluster the average distance between its points is computed. The factor is the ratio of the smallest average to the cluster's own average. It is largest for the tightest cluster. It shrinks for loose clusters. Tight clusters therefore get more points. The reason is how the generation works. Generation interpolates between points. In a tight cluster the points are close. The new point falls in a dense region that belongs to the minority. In a loose cluster the points are far apart. The new point can fall in an empty gap or toward the majority. Generation is therefore directed to the reliable clusters.

The third factor measures the average distance from the cluster to the whole majority. The factor is the ratio of this distance to the largest such distance. It is largest for the cluster farthest from the majority. Far clusters therefore get more points. The reason is the safety of the class boundary. A cluster near the majority sits in the region where the two classes mix. Generation there pushes the synthetic points into the majority area. This blurs the boundary. A far cluster sits in clean minority territory. Generation there strengthens the minority without entering the other class.

The fourth factor measures how connected the cluster is to the rest of the minority. For each cluster the average distance from its center to the other cluster centers is computed. The factor is the ratio of the smallest average to the cluster's own average. It is largest for a cluster near the others. It shrinks for a cluster on the edge. Central clusters therefore get more points. The reason is the same as the single point rule. This time it applies to a group. A cluster far from everyone is already in an extreme region of the class. This cluster can be noise rather than a core part. Feeding it spreads points into the surrounding empty space. This factor completes the third one. The third keeps generation away from the majority. The fourth keeps generation inside the body of the minority.

A cluster gets heavy generation only when it is good on all four criteria. A weakness in any single factor pulls the product down. A very loose cluster is one example. A cluster pressed against the majority or fully isolated is another. This behavior is deliberately cautious. Generation gathers on clusters that are large, tight, far from the majority and connected to the body at the same time.

A single point cluster is treated as an outlier. Its weight is zero. This cluster gets no synthetic points. A lone point with no neighbor can be noise rather than a rare case. The method does not replicate it.

The weights are then normalized to sum to one. Each cluster's share is its own normalized weight. The shares are rounded to whole numbers. The total is set to exactly the required number. Generation then works inside each cluster with its own points. SMOTE or G-SMOTE produces the points assigned to that cluster.

$$w(c) = \left(\frac{n_c}{n_{max}} \right) \times \left(\frac{a_{min}}{a_c} \right) \times \left(\frac{m_c}{m_{max}} \right) \times \left(\frac{o_{min}}{o_c} \right), \quad w(c) = 0 \text{ if } n_c \leq 1$$

$$p(c) = \frac{w(c)}{\sum w(c')}$$

$$count(c) = round(p(c) \times N), \quad N = \text{majority count} - \text{minority count}$$

Symbols: For a set c ,

n_c is the size and $n_{\max}$ are the largest set size,

a_c is the average pairwise distance within the set and $a_{\min}$ is the smallest a_c among all sets,

m_c is the average distance of the set to the entire majority class and $m_{\max}$ is the largest m_c ,

o_c is the average distance of the center of the set to the centers of other sets and $o_{\min}$ is the smallest o_c .

4.4 Implementation

Phase 2 keeps the evaluation structure of Phase 1 unchanged. It only changes how the training set is constructed. A separate training set is constructed for each of the five strategies. In the no oversampling case the training set is used as it is. In the SMOTE and G-SMOTE cases the minority class is treated as one single pool. It is oversampled until the classes become balanced.

In the Cluster+SMOTE and Cluster+G-SMOTE cases the steps explained in Sections 4.1 to 4.3 are applied. The training minority class is divided into clusters. The number of clusters is selected by the elbow method. A point share is assigned to each cluster using the weighting rule. Single point clusters are skipped with zero weight. SMOTE or G-SMOTE is then applied inside each cluster. The synthetic points assigned to that cluster are generated from its own points.

The total number of synthetic points is always equal to the amount needed to raise the minority class to the majority class size. After the training set is constructed the remaining steps are the same as in Phase 1. Each of the five classifiers is run on the constructed training set. For each dataset and classifier the best parameter found in the internal validation of Phase 1 is used. This parameter is either k or radius r . The same parameter is kept fixed across all strategies. The classifier then predicts the unchanged test fold. Error, G-mean and F1 are calculated for each prediction. These values are averaged across the five folds.

The five strategies are compared on the same folds. They use the same classifiers. They also use the same fixed parameters. Therefore the only difference between them is how the training set is constructed. This allows the differences in the results to be attributed only to the oversampling strategy.

4.5 Findings and Conclusion

The Phase 2 results are reported in Tables 5, 6, and 7. These tables compare No Oversampling, SMOTE, G-SMOTE, Cluster+SMOTE, and Cluster+G-SMOTE.

The results confirm the main finding from Phase 1. Oversampling improves the metrics that matter most under imbalance. The average G-Mean increases from 0.7653 without oversampling to values above 0.80 for all oversampling methods. The average F1 score also increases from 0.6770 to at least 0.6946.

Tablo 5. *Average errors of 5 Different Classifiers in 5 Different Setups*

Classifier	No Oversampling	SMOTE	G-SMOTE	Cluster+SMOTE	Cluster+G-SMOTE
KNN	0,1198	0,1402	0,1354	0,1379	0,1309
ModKNN	0,1248	0,1377	0,1368	0,1397	0,1343
FuzzyV1	0,1227	0,1384	0,1348	0,1337	0,1295
FuzzyV2	0,1722	0,1773	0,17	0,1707	0,1675
RadiusKNN	0,1234	0,139	0,1274	0,1393	0,1254
Average	0,1326	0,1465	0,1409	0,1443	0,1375

The error results again show the same tradeoff. No Oversampling gives the lowest average error with 0.1326. This does not make it the best method. It mainly shows the effect of the majority class on error. Oversampling improves minority class detection. This can also increase majority class mistakes. Among the oversampling methods Cluster+G-SMOTE gives the lowest average error. Its average error is 0.1375. This is lower than SMOTE, G-SMOTE, and Cluster+SMOTE. This means that clustering helps G-SMOTE control the error cost of oversampling.

Tablo 6. *Average Geomertic Mean of 5 Different Classifiers in 5 Different Setups*

Classifier	No Oversampling	SMOTE	G-SMOTE	Cluster+SMOTE	Cluster+G-SMOTE
KNN	0,773	0,8397	0,8288	0,8361	0,8297
ModKNN	0,7775	0,834	0,8216	0,8233	0,8235
FuzzyV1	0,7784	0,8426	0,8255	0,8394	0,8293
FuzzyV2	0,774	0,7807	0,7801	0,7744	0,7773
RadiusKNN	0,7239	0,8102	0,7727	0,8056	0,7977
Average	0,7653	0,8214	0,8057	0,8158	0,8115

For G-Mean the best result belongs to SMOTE. Its average G-Mean is 0.8214. Cluster+SMOTE follows with 0.8158. Cluster+G-SMOTE gives 0.8115. G-SMOTE gives 0.8057. This means that clustering does not improve the best G-Mean result. Standard SMOTE remains the strongest method for class balance.

Tablo 7. *Average F1-Score of 5 Different Classifiers in 5 Different Setups*

Classifier	No Oversampling	SMOTE	G-SMOTE	Cluster+SMOTE	Cluster+G-SMOTE
KNN	0,6987	0,7152	0,7239	0,7263	0,7293
ModKNN	0,6991	0,7147	0,7196	0,7167	0,7241
FuzzyV1	0,7027	0,7217	0,724	0,7344	0,7329
FuzzyV2	0,6276	0,6278	0,634	0,6291	0,6333
RadiusKNN	0,6569	0,6937	0,6817	0,7014	0,7034
Average	0,677	0,6946	0,6966	0,7016	0,7046

For F1 the best result belongs to Cluster+G-SMOTE. Its average F1 score is 0.7046. Cluster+SMOTE follows with 0.7016. G-SMOTE gives 0.6966. SMOTE gives 0.6946. This is the strongest positive result of Phase 2. It shows that clustering before oversampling improves minority class prediction quality. The classifier level results support this interpretation. Fuzzy V1 gives the highest F1 score with Cluster+SMOTE. KNN, Modified KNN, and Radius KNN give their highest F1 scores with Cluster+G-SMOTE. Fuzzy V2 remains the weakest classifier. Its values change only slightly across the strategies.

The dataset level results give a more detailed picture. Cluster based methods do not improve every dataset. Their effect depends on the structure of the data. For Ecoli, Haberman, and Parkinson Cluster+G-SMOTE gives the best average F1 score. These datasets show that clustering can help when local minority regions are useful.

For CTO, PageBlock, and Yeast the pattern is different. These datasets are large and highly imbalanced. SMOTE gives the strongest G-Mean in these datasets. Cluster based methods do not fully dominate standard SMOTE. However they often reduce the loss in F1 or error compared with standard oversampling.

PageBlock and Yeast are important examples. Oversampling improves G-Mean clearly. However No Oversampling keeps the best F1 score. This shows that better class balance does not always mean better minority prediction quality. The classifier may detect more minority samples but also produce more false positives. BCWD is already strong without oversampling. The differences between methods are small. Cluster+SMOTE gives the best average result for this dataset. However the improvement is limited because the original classification problem is already easier. Glass also shows a limitation of clustering. G-SMOTE gives the best F1 and G-Mean for this dataset. Cluster based methods do not improve it. This suggests that clustering can be unstable or less useful when the minority structure is small or weak.

The main conclusion from Phase 2 is clear. Clustering before oversampling is useful but not universally superior. It does not beat SMOTE in average G-Mean. However it gives the best average F1 when combined with G-SMOTE. It also gives the lowest average error among the oversampling methods. Therefore Cluster+G-SMOTE is the strongest Phase 2 method when minority class prediction quality is the main goal. (The detailed dataset-level Phase 2 results are provided in Appendix Tables B.1 to B.5.)

5. Conclusion

This project studied imbalance learning with five KNN based classifiers. Ten datasets were used. The methods were evaluated with Error, G-Mean, and F1 score. The study was divided into two phases. Phase 1 compared No Oversampling, SMOTE, and G-SMOTE. The results showed that oversampling improves G-Mean and F1. It also showed that the error rate can increase after oversampling. This is expected in imbalance problems. Oversampling shifts the classifier toward the minority class. This helps minority class detection. It can also create

more majority class errors. Phase 2 added clustering before oversampling. The minority class was first divided into clusters. Then SMOTE or G-SMOTE was applied inside each cluster. The aim was to generate synthetic samples inside local minority regions instead of using the whole minority class as one pool.

The overall results show that oversampling is useful. No Oversampling gives the lowest error. However it gives the weakest average G-Mean and F1. This confirms that error alone is misleading in imbalance learning. G-Mean and F1 give a clearer view of minority class performance. SMOTE gives the best average G-Mean. This means that it is the strongest method for class balance. Cluster+G-SMOTE gives the best average F1. It also gives the lowest average error among the oversampling methods. This means that Cluster+G-SMOTE is the strongest method when minority class prediction quality is the priority.

The dataset level results show that there is no single best method for every dataset. Large and highly imbalanced datasets behave differently from smaller datasets. For CTO, PageBlock, and Yeast oversampling mainly improves G-Mean. However F1 does not always improve. In PageBlock and Yeast No Oversampling keeps the best F1 score.

This shows that better balance between classes does not always mean better minority prediction quality. Oversampling can increase minority recall. It can also increase false positives. When this happens G-Mean improves but F1 may decrease. Smaller datasets show a more positive pattern for cluster based oversampling. Ecoli, Haberman, and Parkinson benefit from Cluster+G-SMOTE in F1. This suggests that clustering is useful when the minority class has meaningful local regions. It gives the oversampler a more careful structure. The results also show the limits of the proposed approach. Clustering is not a guaranteed improvement. It helps in several datasets. It does not help in all datasets. In some cases standard SMOTE remains better. In some cases G-SMOTE remains better.

Overall this project shows three main points. First, oversampling improves imbalance sensitive metrics for KNN based classifiers. Second, Error, G-Mean, and F1 can lead to different interpretations. Third, clustering before oversampling is most useful when local minority structure matters. Its strongest contribution appears when it is combined with G-SMOTE and evaluated with F1.

6. APPENDIX

Appendix A. Dataset-Level Phase 1 Results

Appendix Table A.1. Phase 1 KNN Results by Dataset

Phase 1 KNN									
Dataset	No Oversampling			SMOTE			G-SMOTE		
	Error	G-mean	F1	Error	G-mean	F1	Error	G-mean	F1
BCWD	0,0316	0,9598	0,9563	0,0316	0,9651	0,9576	0,0334	0,9595	0,9543
CTO	0,0301	0,8397	0,748	0,0545	0,9214	0,6762	0,0329	0,8553	0,7397
Ecoli	0,1249	0,7965	0,7106	0,116	0,8469	0,7538	0,1189	0,8296	0,7413
Glass	0,135	0,7002	0,6154	0,1677	0,7644	0,6328	0,1727	0,7844	0,641
Haberman	0,2841	0,4338	0,2889	0,3268	0,6104	0,4532	0,327	0,6548	0,5043
Heart	0,1619	0,7565	0,6806	0,1781	0,8427	0,7431	0,1684	0,8165	0,7241
PageBlock	0,0272	0,9076	0,8621	0,0429	0,9326	0,8116	0,036	0,9265	0,8341
Parkinson	0,0976	0,8457	0,7884	0,1382	0,8769	0,7675	0,1025	0,8595	0,7896
Pima	0,2565	0,6658	0,5803	0,2631	0,7277	0,6494	0,2722	0,71	0,628
Yeast	0,0485	0,824	0,7564	0,0835	0,9088	0,7066	0,0903	0,8916	0,6822
Average	0,1198	0,773	0,6987	0,1402	0,8397	0,7152	0,1354	0,8288	0,7239

Appendix Table A.2. Phase 1 Modified KNN Results by Dataset

Phase 1 ModKNN									
Dataset	No Oversampling			SMOTE			G-SMOTE		
	Error	G-mean	F1	Error	G-mean	F1	Error	G-mean	F1
BCWD	0,0334	0,9573	0,9535	0,0298	0,9654	0,9595	0,0316	0,9608	0,9565
CTO	0,0301	0,8397	0,748	0,0545	0,9214	0,6762	0,0329	0,8553	0,7397
Ecoli	0,1249	0,7965	0,7106	0,116	0,8469	0,7538	0,1189	0,8296	0,7413
Glass	0,14	0,7018	0,6138	0,154	0,7716	0,6517	0,1678	0,777	0,64
Haberman	0,2973	0,4592	0,3063	0,3334	0,6068	0,4491	0,3399	0,6425	0,4881
Heart	0,2052	0,7218	0,6235	0,2087	0,7506	0,6464	0,2086	0,7395	0,6353
PageBlock	0,027	0,9096	0,8638	0,04	0,9367	0,823	0,0356	0,9182	0,8327
Parkinson	0,0722	0,8924	0,8434	0,1076	0,8939	0,8051	0,077	0,8975	0,8415
Pima	0,2669	0,6644	0,5761	0,2552	0,7376	0,6616	0,276	0,6982	0,6139
Yeast	0,0505	0,8321	0,752	0,0775	0,9093	0,7208	0,0795	0,8971	0,7068
Average	0,1248	0,7775	0,6991	0,1377	0,834	0,7147	0,1368	0,8216	0,7196

Appendix Table A.3. Phase 1 Fuzzy KNN Variant 1 Results by Dataset

Phase 1 FuzzyV1									
Dataset	No Oversampling			SMOTE			G-SMOTE		
	Error	G-mean	F1	Error	G-mean	F1	Error	G-mean	F1
BCWD	0,0298	0,9622	0,9588	0,0316	0,9651	0,9576	0,0316	0,9619	0,9568
CTO	0,0301	0,8397	0,748	0,0545	0,9214	0,6762	0,0329	0,8553	0,7397
Ecoli	0,1249	0,7965	0,7106	0,116	0,8469	0,7538	0,1189	0,8296	0,7413
Glass	0,1257	0,7043	0,6279	0,1633	0,7545	0,6261	0,1684	0,7674	0,6366
Haberman	0,3136	0,4698	0,3087	0,3336	0,631	0,479	0,3466	0,632	0,4803
Heart	0,1718	0,7483	0,6675	0,178	0,8334	0,7378	0,1784	0,7936	0,7008
PageBlock	0,027	0,9113	0,8643	0,0418	0,939	0,8177	0,036	0,9265	0,834
Parkinson	0,0822	0,8725	0,8226	0,1224	0,887	0,7909	0,087	0,8781	0,8209
Pima	0,2722	0,6558	0,5655	0,2644	0,7332	0,6572	0,263	0,7166	0,6362
Yeast	0,0492	0,8233	0,7531	0,0788	0,9145	0,7211	0,0849	0,8946	0,6936
Average	0,1227	0,7784	0,7027	0,1384	0,8426	0,7217	0,1348	0,8255	0,724

Appendix Table A.4. Phase 1 Fuzzy KNN Variant 2 Results by Dataset

Phase 1 FuzzyV2									
Dataset	No Oversampling			SMOTE			G-SMOTE		
	Error	G-mean	F1	Error	G-mean	F1	Error	G-mean	F1
BCWD	0,0632	0,9171	0,9088	0,0719	0,9117	0,8981	0,0649	0,9158	0,9065
CTO	0,2474	0,7038	0,2501	0,2611	0,7043	0,2445	0,2507	0,6987	0,2462
Ecoli	0,1427	0,8654	0,743	0,1427	0,8654	0,743	0,1427	0,8654	0,743
Glass	0,1443	0,7053	0,6124	0,1398	0,7198	0,625	0,1256	0,7271	0,6532
Haberman	0,2614	0,5826	0,4433	0,2744	0,5908	0,4478	0,2549	0,6014	0,466
Heart	0,1476	0,8504	0,7664	0,1612	0,8495	0,7573	0,1409	0,8573	0,7804
PageBlock	0,0819	0,7732	0,612	0,0837	0,7682	0,6034	0,0822	0,7764	0,6136
Parkinson	0,2859	0,7599	0,612	0,2756	0,7782	0,631	0,2806	0,7635	0,6171
Pima	0,2735	0,6961	0,6118	0,2761	0,7063	0,6243	0,2735	0,7008	0,6175
Yeast	0,0741	0,8864	0,7161	0,0862	0,9129	0,7035	0,0842	0,8943	0,6964
Average	0,1722	0,774	0,6276	0,1773	0,7807	0,6278	0,17	0,7801	0,634

Appendix Table A.5. Phase 1 RadiusKNN Results by Dataset

Phase 1 RadiusKNN									
Dataset	No Oversampling			SMOTE			G-SMOTE		
	Error	G-mean	F1	Error	G-mean	F1	Error	G-mean	F1
BCWD	0,0456	0,9365	0,9342	0,0438	0,9457	0,9386	0,0439	0,9401	0,937
CTO	0,0301	0,8397	0,748	0,0545	0,9214	0,6762	0,0329	0,8553	0,7397
Ecoli	0,1249	0,7965	0,7106	0,116	0,8469	0,7538	0,1189	0,8349	0,7437
Glass	0,1211	0,7199	0,6512	0,1399	0,7342	0,6479	0,1257	0,7282	0,6551
Haberman	0,2647	0,05	0,0235	0,3041	0,4091	0,2886	0,2745	0,1798	0,1222
Heart	0,2019	0,7443	0,6452	0,2087	0,8085	0,6974	0,1986	0,7902	0,6841
PageBlock	0,028	0,902	0,8572	0,044	0,9378	0,8101	0,0429	0,9319	0,8116
Parkinson	0,1031	0,8435	0,7728	0,1433	0,8593	0,7479	0,0925	0,8753	0,8041
Pima	0,2604	0,5826	0,4914	0,2617	0,7247	0,6464	0,2656	0,6929	0,6086
Yeast	0,0546	0,8239	0,7345	0,0741	0,9146	0,7297	0,0782	0,8985	0,7106
Average	0,1234	0,7239	0,6569	0,139	0,8102	0,6937	0,1274	0,7727	0,6817

Appendix B. Dataset-Level Phase 2 Results

Appendix Table B.1. Phase 2 KNN Results by Dataset

Phase 2 KNN															
Dataset	No Oversampling			SMOTE			G-SMOTE			Cluster+SMOTE			Cluster+G-SMOTE		
	Error	G-mean	F1	Error	G-mean	F1	Error	G-mean	F1	Error	G-mean	F1	Error	G-mean	F1
BCWD	0,0316	0,9598	0,9563	0,0316	0,9651	0,9576	0,0334	0,9595	0,9543	0,0281	0,9688	0,9624	0,0316	0,9609	0,9565
CTO	0,0301	0,8397	0,748	0,0545	0,9214	0,6762	0,0329	0,8553	0,7397	0,0339	0,8911	0,7527	0,0315	0,8601	0,7497
Ecoli	0,1249	0,7965	0,7106	0,116	0,8469	0,7538	0,1189	0,8296	0,7413	0,1189	0,8289	0,7397	0,113	0,8504	0,7623
Glass	0,135	0,7002	0,6154	0,1677	0,7644	0,6328	0,1727	0,7844	0,641	0,1819	0,7603	0,6219	0,1727	0,7528	0,6138
Haberman	0,2841	0,4338	0,2889	0,3268	0,6104	0,4532	0,327	0,6548	0,5043	0,3465	0,6153	0,4584	0,33	0,6366	0,4815
Heart	0,1619	0,7565	0,6806	0,1781	0,8427	0,7431	0,1684	0,8165	0,7241	0,1951	0,8235	0,7187	0,1613	0,83	0,7426
PageBlock	0,0272	0,9076	0,8621	0,0429	0,9326	0,8116	0,036	0,9265	0,8341	0,0387	0,9358	0,8273	0,0338	0,9302	0,8433
Parkinson	0,0976	0,8457	0,7884	0,1382	0,8769	0,7675	0,1025	0,8595	0,7896	0,0921	0,9053	0,8287	0,0975	0,8629	0,7967
Pima	0,2565	0,6658	0,5803	0,2631	0,7277	0,6494	0,2722	0,71	0,628	0,2579	0,7362	0,6605	0,2604	0,7147	0,6339
Yeast	0,0485	0,824	0,7564	0,0835	0,9088	0,7066	0,0903	0,8916	0,6822	0,0863	0,8963	0,6933	0,0775	0,8985	0,7128
Average	0,1198	0,773	0,6987	0,1402	0,8397	0,7152	0,1354	0,8288	0,7239	0,1379	0,8361	0,7263	0,1309	0,8297	0,7293

